

Tối Ưu Hóa Ngân Sách Khuyến Mãi (Voucher Allocation) Trong E-Commerce

Mỗi đợt Mega Sale, sàn TMĐT chi hàng triệu USD cho voucher — nhưng phần lớn ngân sách bị **lãng phí do phân bổ cào bằng**. Data Platform được xây dựng để giải quyết triệt để bài toán này.

Persuadables — Mục tiêu số 1

Chỉ mua hàng **NEU CÓ VOUCHER** → Nhóm duy nhất mang lại Incremental GMV thực sự.

Sure Things — Khách trung thành

Không có voucher **VẪN MUA** → Tặng voucher gây thất thoát biên lợi nhuận trực tiếp.

Lost Causes — Khách thờ ơ

Dù tặng bao nhiêu **CŨNG KHÔNG MUA** → Lãng phí ngân sách hoàn toàn.

Sleeping Dogs — Khách nhạy cảm

Nhận thông báo voucher sẽ cảm thấy bị làm phiền và **HỦY DỊCH VỤ**.

Kiến Trúc Tổng Thể

01

Ingestion Layer

Nhận dữ liệu lịch sử 926K users và luồng Clickstream realtime từ Kafka.

02

Storage Layer (MinIO S3)

Hồ dữ liệu Parquet **bất biến** — Immutable Lakehouse làm nguồn sự thật duy nhất.

03

Transformation Layer (DuckDB + dbt)

Động cơ biến đổi 3 tầng Medallion: Bronze → Silver → Gold.

04

Online Serving Layer (Redis Cluster)

Feature Store 32 shards, đọc sub-millisecond phục vụ inference tức thì.

05

Orchestration & Control Plane (Airflow + PostgreSQL)

Điều phối 8 DAGs và quản trị sổ cái vận hành toàn hệ thống.

Tách biệt Compute & Storage

MinIO lưu trữ, DuckDB tính toán batch, Redis phục vụ đọc nhanh.

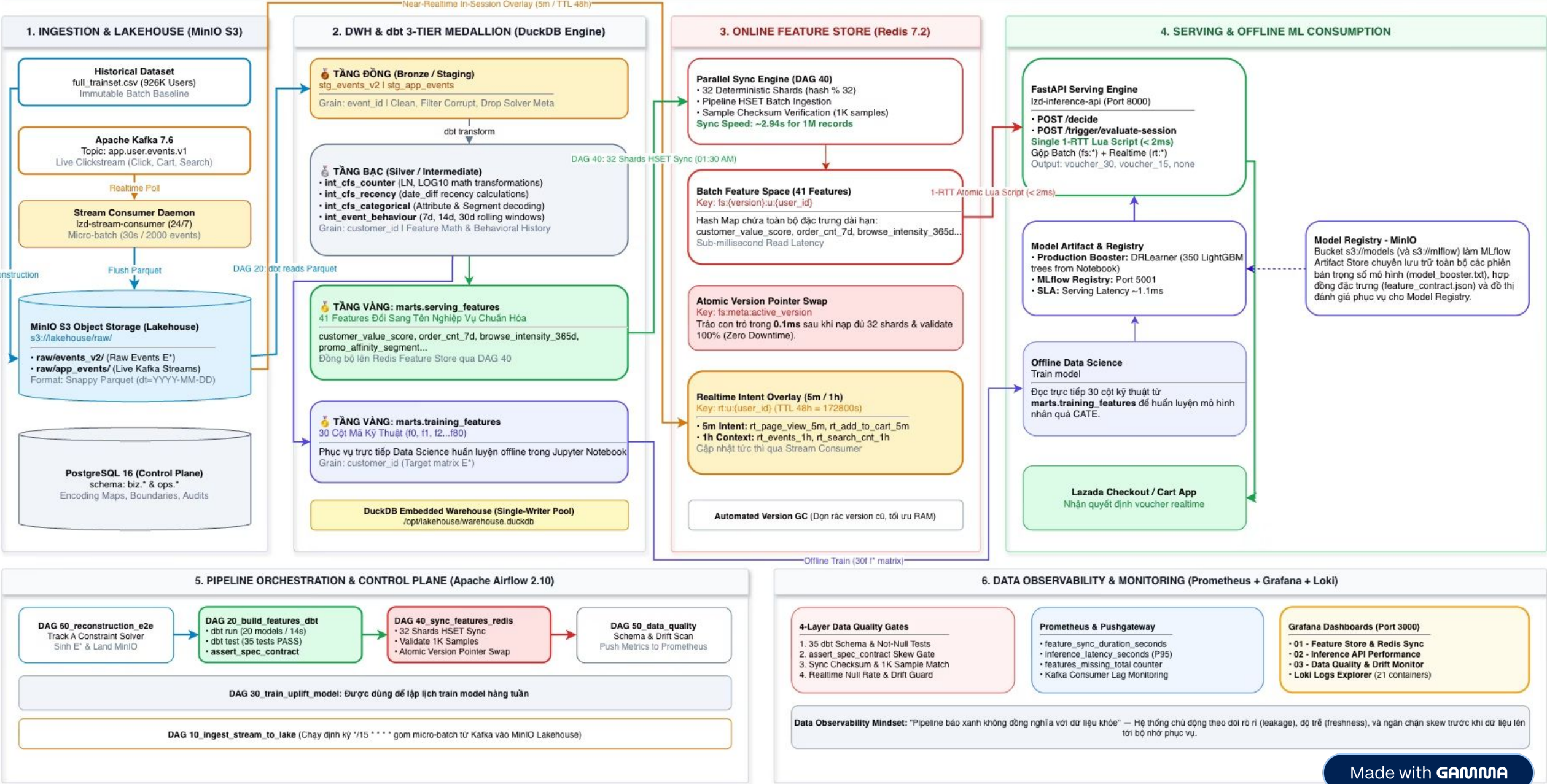
Immutable Source of Truth

Dữ liệu thô không bị sửa đổi — tái lập 100% khi có sự cố.

Zero-Downtime Serving

Cập nhật Feature Store mà không gián đoạn hệ thống đang chạy.

Architecture Data Pipeline



Dữ Liệu Tồn Tại Ở Đâu Và Được Lưu Trữ Như Thế Nào?

Hệ thống áp dụng nguyên tắc "**Đúng việc — Đúng công cụ**" với 3 công nghệ lưu trữ chuyên biệt, mỗi tầng đảm nhiệm một vai trò riêng biệt và không thể thay thế.

Object Storage Lakehouse (MinIO S3)

Định dạng: Snappy Parquet phân vùng theo ngày `dt=YYYY-MM-DD`.

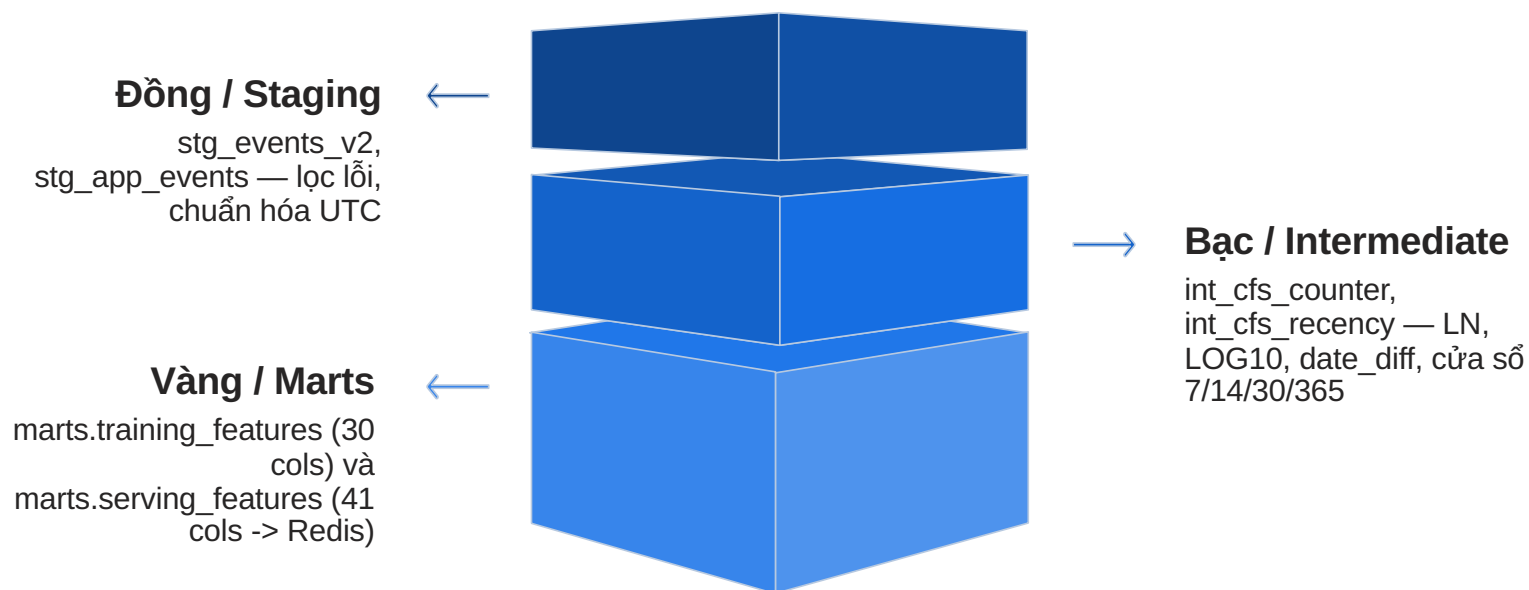
Nhiệm vụ: Lưu sự kiện thô bất biến tại `raw/events_v2/` (lịch sử) và `raw/app_events/` (Kafka).

Bucket: `lakehouse` (dữ liệu phân tích), `models & mlflow` (artifact mô hình).

Control Plane & Metadata (PostgreSQL 16)

Schema

dbt Medallion Architecture: Chuyển Dữ Liệu Sự Kiện Thô Thành Feature Marts



🏆 Tầng Đồng — Staging

stg_events_v2, stg_app_events · Grain: event_id
Lọc sự kiện hỏng, chuẩn hóa timestamp UTC, loại bỏ metadata không hợp lệ.

🥈 Tầng Bạc — Intermediate

int_cfs_counter, int_cfs_recency, int_event_behaviour · Grain: customer_id
Tính LN, LOG10, date_diff, Rolling Windows 7d / 14d / 30d / 365d.

🥇 Tầng Vàng — marts.training_features

Giữ **30 cột mã kỹ thuật** (f0..f80) dành cho Data Science huấn luyện **offline**.

🥇 Tầng Vàng — marts.serving_features

Chuẩn hóa thành **41 cột tên nghiệp vụ** (customer_value_score, browse_intensity_365d, order_cnt_7d...) đồng bộ lên Redis Online Store.

Orchestration & Sync Engine: Điều Phối Tự Động & Đồng Bộ Phân Mảnh 32 Shards

🔧 Điều Phối Luồng Dữ Liệu (Airflow DAGs)

→ DAG 60 — reconstruction_e2e

Giải mã dataset, sinh chuỗi sự kiện thô E* và lưu vào Lakehouse.

→ DAG 20 — build_features_dbt

Kích hoạt **dbt run 20 models & 35 data tests** lúc 01:00 AM.

→ DAG 40 — sync_features_to_redis

Đồng bộ đặc trưng lên Redis lúc 01:30 AM.

→ DAG 10 — ingest_stream_to_lake

Chạy mỗi giờ — Compaction Parquet nhỏ & giám sát luồng Kafka.

⚡ Kỹ Thuật Đồng Bộ Phân Mảnh (Redis Sync)

1. Deterministic 32-Shards Sync

Băm `hash(user_id) % 32`, nạp song song qua pipeline HSET trong **2.94 giây** cho gần 1 triệu user.

2. Data Integrity Checksum

Task `validate_sync` chọn ngẫu nhiên **1,000 bản ghi** đối soát 1-1 giữa DuckDB và Redis — Value Diff = 0, Checksum Matching 100%.

3. Zero-Downtime Atomic Version Swap

Nạp vào `fs:v20260801:u:*`, tráo con trỏ `fs:meta:active_version` trong **0.1ms** sau khi nạp đủ 32 shards.

4. Realtime In-Session Overlay

Key `rt:u:{user_id}` (TTL 48h) lưu hành vi 5 phút (`rt_page_view_5m`, `rt_add_to_cart_5m`) do Stream Consumer cập nhật tức thì.

Hệ Thống Giám Sát 4 Lớp

1 dbt Data Tests

35 bài test tự động kiểm tra `not_null`, `unique`, `accepted_values` trên toàn bộ staging và marts.

2 Data Contract Skew Gate

Task `assert_spec_contract` đối chiếu **44 cột DuckDB** với `feature_spec.yml` — **thiếu 1 cột** → **chặn đứng pipeline ngay lập tức** để chống Training-Serving Skew.

3 Sync Checksum Validation

Đối soát dữ liệu offline vs. online store trước khi kích hoạt phiên bản mới.

4 Real-time Dashboards (Grafana + Prometheus)

Thu thập metrics, hiển thị tỷ lệ Null, Freshness và CPU/RAM của **21 container**.

Case Study thực tế: Khi selector loại trừ model, bảng serving bị thiếu nhưng pipeline dễ bị bỏ qua nếu chỉ check task status. Giải pháp: Hard-Fail tại